Report

AI/ML-012

- **Team Details**:

Team leader:

1.) Siddhi Borase
Roll no.: 240002072
GitHub: https://github.com/ssb120107
LinkedIn: https://www.linkedin.com/in/siddhi-borase-0b0353351/

Other team members:

2.) Aarushi Pandey
Roll no.: 240003002
GitHub: https://github.com/aarushipd12
LinkedIn: https://www.linkedin.com/in/aarushi-pandey-2a8678320

3.) Shriya Deo
Roll no.: 240041013
GitHub: https://github.com/Shriya-1807
LinkedIn: https://www.linkedin.com/in/shriya-deo-396581334

4.) Tanishka Sihara
Roll no.: 240003078
GitHub: https://github.com/tanishka-sihara
LinkedIn: http://www.linkedin.com/in/tanishka-sihara

- **Team**: AI/ML-012
- **PS Name**: Drone Footage Object Detection

- **Project Introduction**:

1. **Aim** of the project was to develop a computer vision system for detecting and classifying objects in drone footage to support real world applications. Our project is built mainly focusing to support urban planning, traffic monitoring, tracking targets (security concerns), etc.

2. **Tools and Technology used:**

Tech used	**Used it for**
YOLOv8s Worldv2	Model used for training on VisDrone dataset
YOLOv8l Worldv2	Pre-trained weights used for text-prompt based detection
SAHI (Slicing Aided Hyper Inference)	Inference
DeepSORT	Tracker
Streamlit	Dashboard

Table 1: Tools and technology used

- **Project Workflow**:

The project workflow that was followed by the team:

1. *Learning:*
   (a) Learning concepts and their applications, from various sources.
   (b) Usage of essential python libraries such as Pandas, Numpy, Matplotlib.
   (c) Exploring key steps and procedure to build the project.
   (d) Basics of PyTorch.

2. *Data Collection:*
   The online websites containing sources for the datasets were provided in the problem statement. After exploring and searching among the datasets, VisDrone2019 dataset was found to be the most suitable for object detection. It had pre-split folders as train, test, and validation data. Each folder contained annotations corresponding to each frame(in the case of videos dataset), or image(in the case of images dataset).

3. *Data Analysis:*
To begin with coding, it was essential to understand the data storage hierarchy. As mentioned, a total of 6 folders were downloaded: train, val, and test folders for images and similarly for video footage.

   (a) Each of the folders in image format, consists of 2 sub folders- 'annotations', 'images'. The 'annotations' sub folder consisted of annotation text files corresponding to each image stored in the 'images' sub folder.

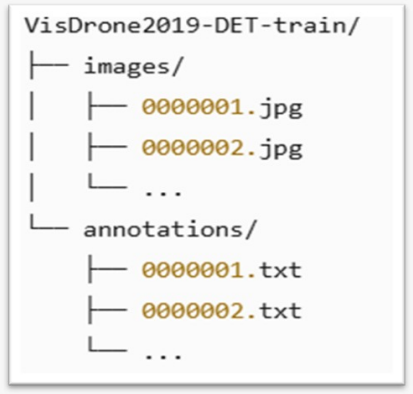

Figure 1: Dataset Hierarchy of image dataset folders

   (b) Each of the folders in video format, consists of 2 sub folders- 'annotations', 'sequences'. The 'sequences' sub folder consists of multiple sub folders for multiple video footage, each of which contains the frames as (frame-name).jpg files. The 'annotations' sub folder consisted of annotation text files corresponding to each frame stored in each of the video footage sub folders.

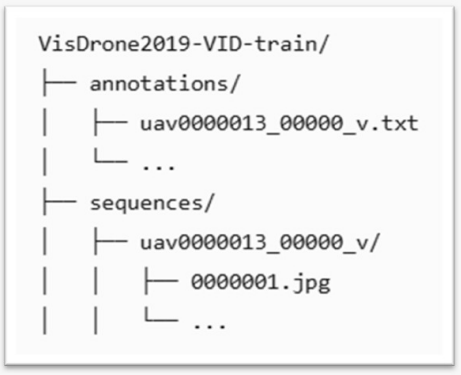

Figure 2: Dataset Hierarchy of video footage folders

4. *Data Pre-processing:*

   (a) Preprocessing the data involves transforming raw, inconsistent, and noisy information into a clean and structured format suitable for subsequent model training. For this project, the YOLO model specifically required annotations of each training image to be converted from the original VisDrone format to the YOLO format.

   (b) And, this model also requires the annotations to be converted to a specific format suitable for yolo model implementation. These requirements were satisfied during data pre-processing.

   (c) The original annotations for each instance of each image were of the form:
   x, y, width, height, score, category, truncation, occlusion

   These were converted to the YOLO-specific format:
   yolo-cls, x-center, y-center, norm-width, norm-height

   ,where the center coordinates and dimensions were normalized relative to the image size.

   (d) The newly generated annotation files for each training image were saved in a dedicated folder named 'labels' (and similarly for validation images). Additionally, a cleaning function was implemented to remove any annotation lines where the class index exceeded the

defined maximum class index, ensuring only relevant labels were retained for training.

5. *Train-test splitting:*

   Every Machine Learning project requires its data to be split into train data, and test data (split by the ratio 80:20, by default).

   The VisDrone2019 dataset, that has been used for training, already had separate folders containing distinct training, testing, and validation data. Hence, this step was not required.

6. *Model Training:*

   Model training was conducted using Google Colaboratory, leveraging its GPU capabilities to efficiently run training for 100 epochs. Since the VisDrone dataset's video folders already contained extracted frames, training was performed exclusively on the image dataset. The trained model could then be directly applied to video frames.

   Key steps in the training process included:

   (a) Configuration via YAML file:
   A YAML configuration file was created to manage critical parameters and settings for training. This file defined the 10 object classes to be detected, along with their corresponding class IDs. The target classes included: pedestrian, people, bicycle, car, van, truck, tricycle, awning-tricycle, bus, and motor.

   (b) Training loop:
   The YOLOv8s-worldv2 model was trained on the image dataset for up to 100 epochs. Early stopping with a patience of 10 epochs was incorporated, allowing the training process to terminate if no improvement in validation metrics was observed for 10 consecutive epochs. The training halted early at epoch 82 due to this criterion, ensuring efficient use of computational resources.

7. *Evaluation*:

   The model was trained and evaluated on the test dataset. From the evaluation metrics, it was inferred that the model had a fair accuracy level, and was on the higher side of the precision score that most YOLOv8 models achieve.

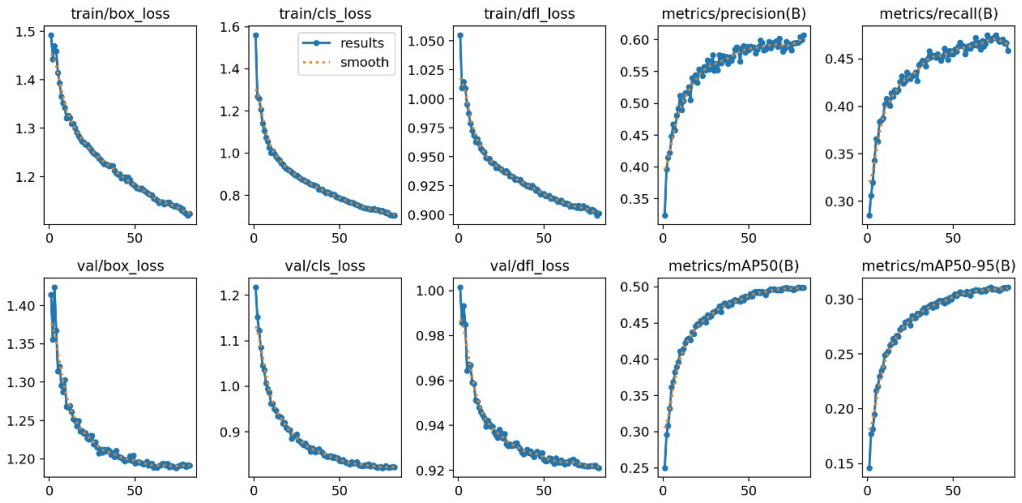

Figure 3: Graphs representing Evaluation metrics and their improvements with increasing epochs

8. *SAHI for Sliced Inference (Slicing Aided Hyper Inference),* ***for Images***

   (a) SAHI is a state-of-the-art library designed to optimize object detection performance on large-scale, high-resolution images. It works by partitioning images into smaller, manageable slices, running detection on each slice independently, and then merging the detection results back into a cohesive output.

   (b) This slicing approach is particularly effective in detecting small or densely packed objects, which are often challenging to detect accurately in full-scale images. By processing smaller slices, SAHI enhances detection precision and recall, making it ideal for datasets like VisDrone where small object detection is critical.

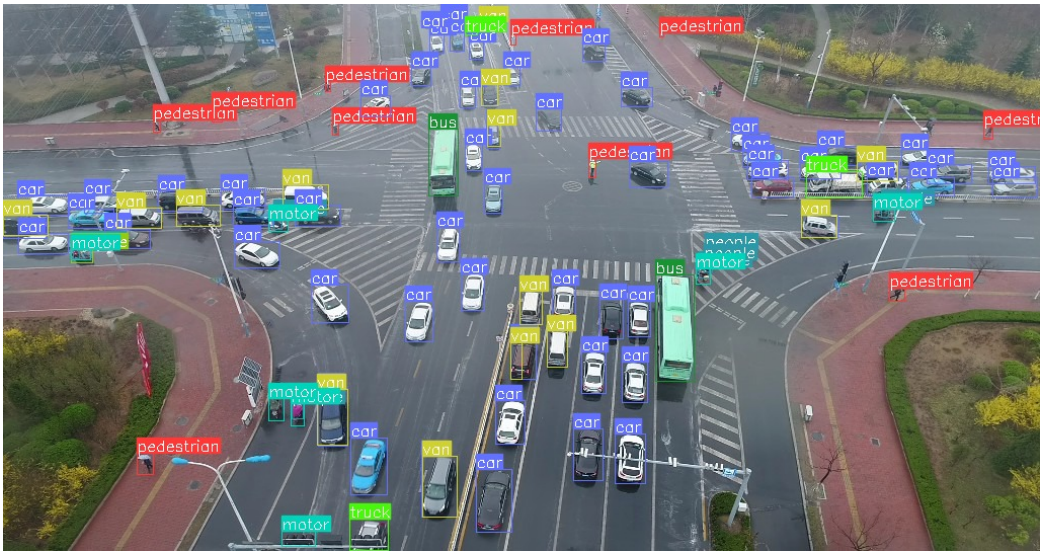

Figure 4: Object Detection using SAHI (Slicing algorithm for inference)

9. *Frame Extraction and DeepSORT Tracker* ***for Video*** *(Deep Simple Online and Realtime Tracking)*

   (a) Since the model was trained exclusively on images, video inputs require a preprocessing step to extract individual frames. A dedicated frame extraction function was implemented to convert user-supplied video files into sequences of images for inference.

   For robust multi-object tracking across video frames, the DeepSORT tracker was integrated. DeepSORT leverages both motion prediction and appearance features to:

   i. Predict the next position of tracked objects using a filter.

   ii. Maintain consistent object IDs across frames by associating detections over time.

   iii. Track multiple objects simultaneously, even under occlusion or crossing paths.

   This integration ensures reliable, real-time tracking performance when analyzing video data, complementing the object detection pipeline effectively.

10. *Dashboard:*

    (a) Objective of the Dashboard:

    The dashboard is aimed to provide an easy-to-use web interface for object detection and tracking in aerial (drone) imagery and video. It enables users to upload drone images or videos and view detection results with bounding boxes and class labels.

    It supports both-

    i. default object detection using YOLOv8s Worldv2, and
    ii. text-prompt based custom detection using YOLOv8l Worldv2.

    (b) User Interface (UI) Design:

    i. Dashboard is built and deployed with Streamlit.
    ii. It includes two processing nodes, displayed as two tabs-
        - Image Processing
        - Video Processing
    iii. Sidebar:
        - Model Selection: Default versus Text-Prompt based detection
          -Default Detection:
          Uses YOLOv8s-Worldv2 model (trained on VisDrone dataset) for object detection on VisDrone classes.
          -Text-prompt Detection:
          Enables user-defined detection targets by accepting a comma-separated list of class names (e.g., "red car, street lights") and configuring pretrained yolov8l-worldv2 to detect only those.
    iv. Output Display:
        - Processed image with object detection is displayed on dashboard (preview is available).
        - To view the processed video (with object detection and tracking), download the video file to the local system, and view.

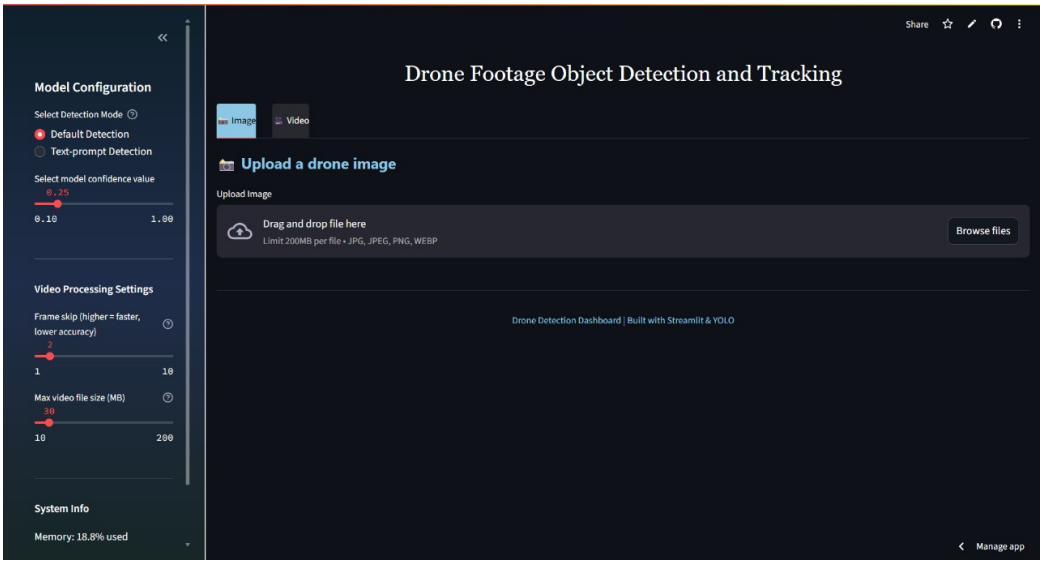

Figure 5: Dashboard Preview for images tab

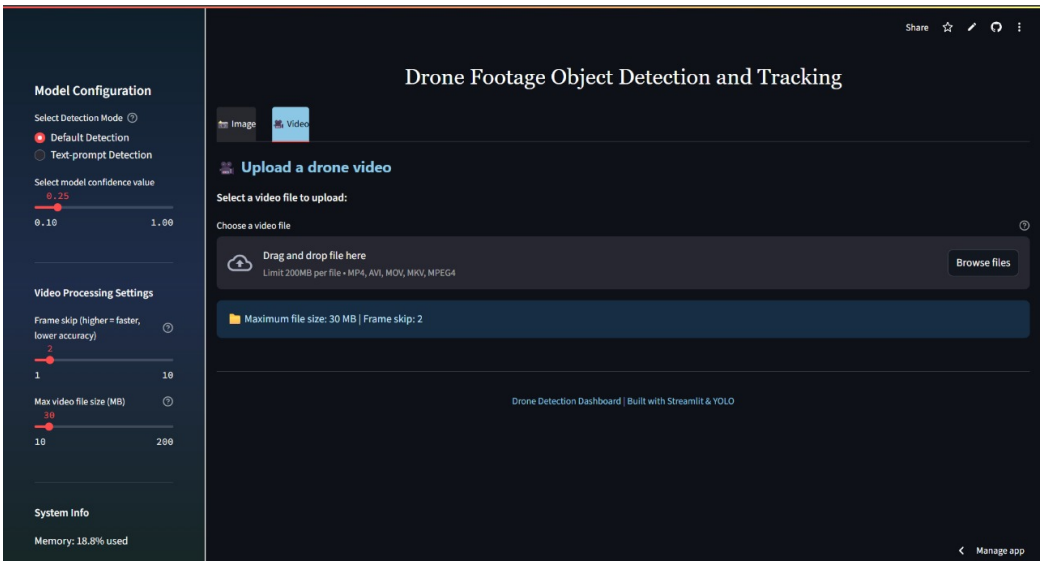

Figure 6: Dashboard Preview for video tab

- **Bonus Features**:

1. **Smart Alert System:**

   This feature allows user to detect any persons present in a defined restricted zone. This restricted zone is set by pointing to the top-left and bottom-right coordinates of the bounding box, from the drone footage imagery. When a person is detected in a restricted zone, an alert is triggered and alarm is generated.

   If a hardware is integrated to the code, the buzzer can ring the alarm and alert the persons/ responsible authority.

Figure 7: Restricted box is defined.

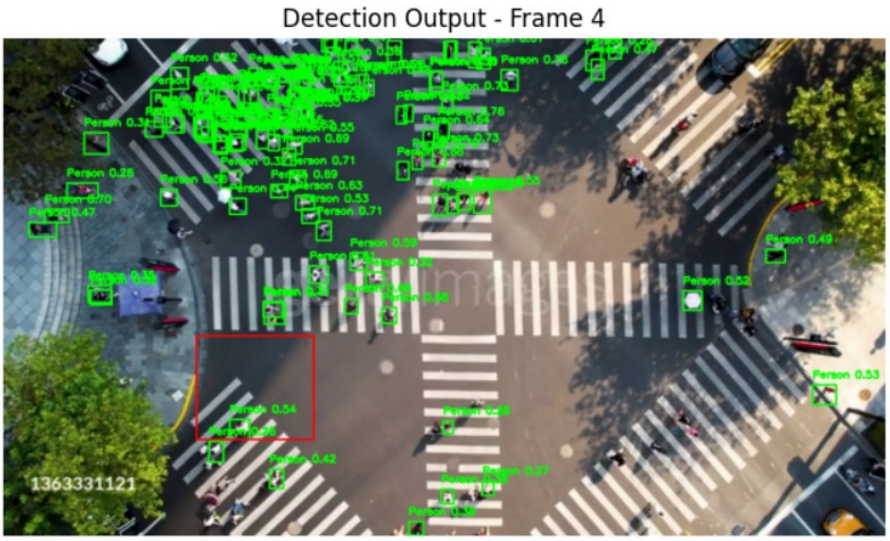

Figure 8: Person detected within restricted zone, and alarm is triggered

2. **Text-prompt based detection:**

   The selected YOLOv8s-Worldv2 model offers text-prompt based detection. It allows user to input a text prompt, and detects the object according to the text-prompt entered by user.

   Traditional object detection models require predefined class lists and extensive retraining for new categories, which limits their flexibility. Open-vocabulary detection addresses this by allowing dynamic specification of target objects using natural language, making models applicable across a wider array of real-world tasks with minimal reconfiguration.

   For its implementation in the project, publicly released YOLOv8s-Worldv2 weights (yolov8s-worldv2.pt) were used, which were already trained on large-scale datasets to capture a broad spectrum of visual and semantic features.

Figure 9: Text prompt entered from user's end: yellow car, black car, white car

- **Practical Applications fit for this project**:

1. The overall project can have extensive application and practical usability in traffic monitoring, and urban planning. By detecting number and type of vehicles on a road will be useful in monitoring traffic.

   If the number and type (in terms of weight) of vehicles passing a particular road is monitored, and it is inferred from the pattern that heavier vehicles (eg. trucks, buses, etc) use the road more often, causing traffic jams, the authorities can plan a connecting bridge construction, or new lane building, toll planning etc. and hence, the model would help in efficient urban planning.

2. Text-prompt based detection: It can be very useful in spotting a particular vehicle in a moving traffic.

   For instance, a criminal is escaping from the police and the only known information is the color of the car in which the criminal was seen escaping (let's say, red), and the road taken by the criminal. This way,

drones can be deployed across the route, and using text prompt based detection for 'red car', Hence, it can serve as an important and useful application for security and other such concerns.

3. Smart Alert System: This feature can have multiple use-cases. In case of repair work of the road- cementing, spreading tar, pipeline work, etc.- the area can be set as a restricted zone so as to avoid people from entering the zone (other than physically barricading). In case, a person still enters the zone, the alert system will be triggered.

   The code can also be integrated with suitable hardware so that a buzzer/ alarm rings whenever the alert system is triggered.

   Another prominent application can be in traffic signal monitoring. The drone camera can be set at a place such that it captures the traffic signal and the vehicles crossing the signal. It can be noted that as the footage frame detects a red signal, a restricted zone will be created ahead of the line behind which the vehicles halt. If a vehicle tries to violate the traffic signal and enters the restricted zone, the alert system will be triggered and a buzzer will ring. As soon as it stops detecting the red signal, and detects green signal, the restricted zone will vanish and no buzzer will ring until the next series of red-green signal.

- **Conclusion**:

1. YOLOv8s-Worldv2 model is deployed to the dashboard, making it ready-to-use with accessible interface for researchers and community users to detect, track and classify objects from drone footages. The dashboard will require user to input a file (image/ video, depending on user's requirements), process the file and output the same image/video with detection, tracking (in case of videos) as well as classification of the objects.

2. In case the user selects the Text-Prompt based detection model, it is deployed with the large version of YOLOv8l-Worldv2, as pre-trained weights have been used. Hence, using a larger model will provide better accuracy results without consuming and exhausting resource constraints. The user will be required to input the image/ video file and enter the text-prompt, and processed file will be returned to the user, with detection and classification.

3. ***APP LINK:*** `https://dashboarddronefootageobjectdetection.streamlit.app/`

- **Key Learnings and Insights**:

1. General workflow and working of an ML-based project.
2. Data Handling with such huge dataset.
3. We tried and explored 4 different models. Among them YOLOv8s-Worldv2 worked most suitably and with better features. Other models we tried were- YOLOv8n, YOLOv8s, SAM2. The reason for dropping these 3 models are as listed below:
   (a) The two YOLOv8 models were dropped because we figured out working with an advanced version of the model, which was the YOLOv8-Worldv2 model. It offered extra features such as zero shot generalization, with text-prompt based detection.
   (b) SAM2 (Segment-Anything-2) is a segmentation-based advanced model. This model was chosen as it provided pixel-wise segmentation and masking. Due to model complexity and storage requirements, launching the training was posing difficulties and errors. The preprocessing step was completed, masks and json file were created, checkpoints and configuration file was downloaded, the official GitHub repo cloned, but during launch of the training loop, some difficulties were faced. Hence, for optimizing limited computational resources, we switched to YOLOv8s-Worldv2 model.

References/ Resources

- VisDrone Dataset: `https://github.com/VisDrone/VisDrone-Dataset`
- Ultralytics official documentation: `https://docs.ultralytics.com/models/yolov8/`
- SAHI Inference: `https://docs.ultralytics.com/guides/sahi-tiled-inference/`
- Streamlit Documentation: `https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://docs.streamlit.io/`

- Ultralytics Documentation: Live Inference with Streamlit Application using Ultralytics YOLO11: https://docs.ultralytics.com/guides/streamlit-live-inference/
- SAM2: Segment Anything Model 2: https://docs.ultralytics.com/models/sam-2/
- Research Paper for Alert System: https://ijcrt.org/papers/IJCRT2101455.pdf
- Reference project on LinkedIn: https://www.linkedin.com/posts/franklin-de-oliveira_computervision-yolo-objectdetection-activity